

Lab 2.b demo walkthrough — the guardrail, step by step

The instructor's build-along demo ("Seven steps — build along on screen"), written down. Use it to replay the build at your own pace, to catch up if you missed the session, or to fix a step that didn't land live. Same rule as the 2.a walkthrough: this is the instructor's own script — there is no secret version.

This covers **the demo guardrail only** (the read-only `db.json` rule). Your own clause — the one your group derived in 2.a — is the exercise; this document deliberately doesn't build it for you.

Before you start: your own copy, branch `feat/governance`, the 2.a docs committed, and the Claude Code **Hooks** lesson done.

Step 1 — Make the file exist

```
npm run reset-db
```

`db.json` is generated and git-ignored — a fresh copy doesn't have it, and a guardrail protecting a file that isn't there is hard to verify.

Step 2 — Layer 1: the instruction

One line in `CLAUDE.md`'s Don't section:

```
- Don't: edit `server/data/db.json` directly — it is generated; change data through the API, or edit seed.json via PR and run `npm run reset-db`
```

This informs. It does not enforce — that's the point of the next two layers.

Step 3 — Layer 2a: the deny rules

Merge into the existing `deny` array in `.claude/settings.json` (never paste over the file — your seed rules and hooks block must survive):

```
"deny": [
  "Edit(server/data/seed.json)",
  "Write(server/data/seed.json)",
  "Edit(server/data/db.json)",
  "Write(server/data/db.json)"
]
```

Step 4 — Layer 2b: the hook

Create `.claude/hooks/protect-db.js`. Same plumbing as `protect-seed.js` (read it first — the four-part message is the pattern):

```
#!/usr/bin/env node
let raw = '';
process.stdin.on('data', (c) => (raw += c));
process.stdin.on('end', () => {
  let filePath = '';
  try {
    filePath = JSON.parse(raw).tool_input?.file_path ?? '';
  } catch {
    process.exit(0); // unparseable – never block unrelated work
  }
  const path = filePath.replaceAll('\\', '/'); // Windows-safe

  if (path.endsWith('server/data/db.json')) {
    console.error(
      [
        'BLOCKED: db.json is generated, not edited.',
        'Why: hand-edits are lost on the next reset and are invisible to everyone else.',
        'Instead: change data through the API, or edit seed.json via PR and run npm run reset-db.',
        'Done means: your data change survives a reset.',
      ].join('\n'),
    );
    process.exit(2); // PreToolUse -> blocks
  }
  process.exit(0);
});
```

Note the message shape — **BLOCKED / Why / Instead / Done means**. A guardrail that only says “no” makes the agent guess, and it guesses creatively.

Step 5 — Register it, then start a NEW session

Add a second entry to the **existing** `PreToolUse` array (don't create a second `hooks` block):

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [
        { "type": "command", "command": "node
\\$CLAUDE_PROJECT_DIR/.claude/hooks/protect-seed.js\""},
        { "type": "command", "command": "node
\\$CLAUDE_PROJECT_DIR/.claude/hooks/protect-db.js\""}
      ]
    }
  ]
}
```

Then **start a new Claude Code session** — hooks load at session start. “My hook never fires” is, nine times out of ten, this step skipped.

Step 6 — Verify the BLOCK

In the new session, ask Claude to do the violation on purpose:

```
Open server/data/db.json and change one todo's title directly in the file.
```

You should get a refusal carrying your hook's message. **Save the refusal text** — pasting it into your PR description is the evidence standard. Then confirm the file is untouched (`git status` shows no change to `db.json` — though being git-ignored, check with `ls -la` or just re-open it).

Step 7 — Verify the ALLOW, then document the bypass

The half everyone skips:

```
Add a one-line comment to server/src/repositories/tags.repository.ts, then run npm
test -w server.
```

No refusal, no noise, suite green — the guardrail permits legitimate work. If this fails, the guardrail is worse than nothing: it now blocks work, and someone will remove it by Friday.

Finally, write the bypass down (in `CLAUDE.md`, next to the Don't line):

Bypass: seed/data changes go through a PR flagged to the instructor – never by disabling the hook.

Common wobbles

Wobble	The move
Hook never fires	New session started? Hooks load at session start. Then: <code>matcher</code> says <code>Edit\ Write ?</code> Path check survives Windows backslashes?
Hook fires on the wrong files	Your path test is too broad — <code>endsWith('server/data/db.json')</code> not <code>includes('db.json')</code> , or you'll catch fixtures
The deny rule fires before the hook	Expected — layers are redundant on purpose; the deny is cheap, the hook explains. Test the hook by removing the deny <i>temporarily</i> , then restore it
Two hooks blocks in settings.json	Merge them — the file wants one <code>PreToolUse</code> array with two command entries
“It worked” with nothing to show	Not verified. The refusal text in your PR description is the standard